Campus Store Inventory & Sequential Checkout

Complete Technical Documentation, Function-by-Function Reference & User Guide

Project: College OJT Retail System	**Architecture:** 2-Tier Modular Vanilla JS
Live Website: https://naveenjaay55-cpu.github.io/OJT/	**GitHub:** https://github.com/naveenjaay55-cpu/OJT
Data Storage: Browser LocalStorage (No DB needed)	**Stretch Goal:** Python 3 Sequential Replay Script

1. System Overview & Architecture

The Campus Store Inventory & Sequential Checkout web application is an enterprise-grade retail POS and inventory management prototype designed for campus stores. The application enforces a clean separation between presentation (DOM Manipulation & UI in js/app.js), business logic and security authorization (Service Layer in js/services.js), and persistence (Browser localStorage). Additionally, a standalone Python 3 verification script (replay_checkout.py) audits sequential transactions and asserts strict inventory integrity.

Layer	File	Core Responsibility
Presentation (UI)	index.html / .html css/styles.css js/app.js	Renders POS cards, inventory tables, checkout form, receipt modals, toast alerts, steppers, and handles DOM user events.
Service & Security	js/services.js	Enforces the Section 9 Service Contract, executes the 11-step sequential checkout pipeline, enforces role security rules in JS logic, and guards invariants.
Persistence	localStorage	Maintains campus_store_products, campus_store_transactions, and campus_store_settings across page reloads and browser sessions.
Stretch Goal Audit	replay_checkout.py	Simulates offline sequential replay from sample-data.json, validating stock availability and testing 'assert product[stock] >= 0'.

2. Dual-Role Simulation & In-Depth Permission Enforcement

A key requirement of this project is that role-based permissions **cannot simply rely on hiding buttons**. The system enforces permissions directly inside the JavaScript application logic. If an unauthorized role invokes a restricted function programmatically, the function explicitly rejects the call and returns an error response.

Role Identity	Allowed Capabilities	Enforced Restrictions	JS Enforcement Point
Cashier (Terminal C001)	• View product catalog & stock • Select items for checkout • Enter purchase quantities • Complete sequential checkouts • Print transaction receipts	• CANNOT add new products • CANNOT edit product info • CANNOT alter stock levels • CANNOT checkout > stock	addProduct() & updateProduct() reject execution immediately with HTTP-like failure response if current role is Cashier.

Store Operations Staff (ID S001)	• Catalog new products • Update pricing & names • Adjust stock quantities • Set low-stock alert limits • Delete obsolete inventory	• CANNOT complete customer checkout transactions at POS terminal	processCheckout() rejects execution immediately if current role is Store Staff, demanding switch to Cashier.

3. One-to-One Feature Breakdown (Requirements Traceability)

Each requirement from the project specification has been individually engineered, validated, and mapped:

FR-01: Product Catalog & Stock Display [Must Have]

Displays all store inventory with Product ID, Name, Category, Unit Price, Available Quantity, and Stock Status. Available in responsive interactive visual cards and a filterable management table.

FR-02: Add Product [Must Have]

Store Operations Staff can add new inventory items with unique Product IDs, names, categories, unit prices, initial stock, and low-stock thresholds. Validates ID uniqueness and positive numbers.

FR-03: Update Product & Restock [Must Have]

Store Operations Staff can modify any existing product details, update prices, and restock quantities. Automatically updates the modified timestamp (updatedAt).

FR-04: Select Product for Checkout [Must Have]

Cashiers can select items for checkout in two intuitive ways: clicking directly on visual product cards in the POS grid, or picking from the quick-select dropdown list.

FR-05: Enter Purchase Quantity [Must Have]

Cashiers can input purchase quantity with numeric controls or interactive decrement (-) and increment (+) steppers. Enforces whole positive integers (> 0).

FR-06: Pre-Purchase Stock Validation [Must Have]

The system compares the requested quantity against the currently available inventory before processing. If requested > available, checkout is blocked immediately.

FR-07: Dynamic Live Total Calculation [Must Have]

Calculates Total Amount = Unit Price × Quantity in real time as the user types or clicks stepper buttons before checkout confirmation. Formatted to two decimal places.

FR-08: Complete Sequential Checkout [Must Have]

Processes the purchase sequentially and atomically. Emits an immutable transaction record with a unique ID, cashier ID, timestamp, and creates a pop-up confirmation receipt.

FR-09: Automatic Stock Decrement [Must Have]

Upon successful checkout, the purchased quantity is immediately deducted from product inventory. All UI cards, tables, badges, and summaries refresh in real time.

FR-10: Strict Oversell Prevention [Must Have]

If requested purchase quantity exceeds available stock, checkout fails with a clear warning: 'Insufficient stock available (Available: X, Requested: Y)'. Crucially, stock remains untouched!

FR-11: Visual Low-Stock Highlighting [Must Have]

Products with available stock <= configured lowStockThreshold are visually distinguished with amber warning tags, animated pulsing dots, and highlighted table rows.

FR-12: Browser LocalStorage Persistence [Must Have]

All product inventory, settings, and completed transactions persist in browser localStorage under designated keys. Data survives browser reloads, restarts, and closures.

FR-13: Sales Analytics & Summary Dashboard [Good to Have]

Top KPI metric cards and an Analytics tab track total sales revenue, completed order count, total units sold, today's sales, average order value, and category revenue share.

FR-14: Historical Transaction Audit Log [Good to Have]

Complete sequential audit log recording every completed checkout with Transaction ID, Date/Time, Product Name & SKU, Quantity, Unit Price, Total Paid, Cashier ID, and Status.

4. Function-by-Function Technical Reference

This section explains in detail **which JavaScript and Python function does which work** across the codebase.

4.1 Service Layer Functions (js/services.js)

The Service Layer implements the Section 9 Service Contract. It manages the in-memory cache, interacts with localStorage, enforces role-based restrictions, and implements business rules.

loadData()

Loads persisted JSON data from localStorage keys (campus_store_products, campus_store_transactions, campus_store_settings). If first time or empty, automatically initializes with baseline sample data (P001 to P004).

saveData()

Serializes and persists in-memory product arrays, transaction history, and settings into localStorage. Returns success/failure status.

resetToSampleData()

Wipes modified storage and restores pristine official sample data (Notebook P001, Blue Pen P002, Water Bottle P003, Chips Packet P004) and baseline transaction T001.

getCurrentRole()

Returns the active simulated role ('cashier' or 'store_operations').

setCurrentRole(role)

Switches and persists the active role. Validates that the role string is either 'cashier' or 'store_operations'.

getActiveCashierId()

Returns current cashier identifier string (default: 'C001').

getActiveStaffId()

Returns current store staff identifier string (default: 'S001').

getProducts()

Returns a deep-cloned array of all products currently in store inventory. Prevents external mutation of internal cache.

getProductById(productId)

Finds and returns a single product object matching the given SKU/ID, or null if not found.

getLowStockProducts()

Filters and returns an array of all products whose stock is at or below their lowStockThreshold (FR-11).

addProduct(product)

Role Enforced (Store Staff Only): Validates that the caller is Store Operations Staff. Validates that productId is unique, non-empty, and price/stock/threshold are non-negative numbers. Adds product to cache, saves to localStorage, and returns result.

updateProduct(productId, updates)

Role Enforced (Store Staff Only): Validates Store Staff role. Finds target product, applies validated updates to name, category, price, stock, or threshold, updates updatedAt timestamp, and persists to localStorage.

deleteProduct(productId)

Role Enforced (Store Staff Only): Removes an item from store inventory. Validates Store Staff role and updates localStorage.

processCheckout(productId, quantity, cashierId)

Role Enforced (Cashier Only) & 11-Step Sequential Pipeline:

1. Checks caller is Cashier.
2. Finds product by productId in inventory.
3. Verifies product existence.
4. Validates quantity is a positive whole integer (> 0).
5. Compares requested quantity against available stock.
6. **If insufficient stock:** Rejects checkout immediately; leaves stock 100% untouched; returns failure response.
7. If sufficient: Computes totalAmount = price * quantity.
8. Decrements product stock by quantity.
9. Generates transaction object (T001, T002...) with timestamp and cashier ID.
10. Persists updated inventory and transactions to localStorage.
11. Returns standard success response with transaction details and remaining stock.

getTransactions()

Returns a deep-cloned array of all recorded checkout transactions in reverse chronological order (latest first).

getSalesSummary()

Aggregates financial KPIs: total revenue, completed order count, units sold, today's sales, average order value, low-stock count, and out-of-stock count.

getSettings() & updateSettings(newSettings)

Retrieves and updates store-level configuration (store name, currency symbol, default thresholds).

4.2 UI Controller Functions (js/app.js)

The UI Controller handles user interaction, event listeners, DOM rendering, modal dialogs, and connects the user interface to the Service Layer.

initApp()

Main initialization routine. Instantiates Bootstrap 5 modal objects (productModal, receiptModal, resetModal), binds all DOM event listeners, synchronizes role styling, and executes initial render of all tabs.

setupEventListeners()

Attaches reactive DOM listeners: role selector dropdown change, POS search and category filter inputs, product dropdown selection, quantity +/- steppers, checkout button, oversell demo button, inventory filters, and modal form submissions.

syncRoleUI()

Updates the visual role indicator in the top navbar and renders the dynamic Role Notice Card. Adjusts button states based on whether the active user is Cashier or Store Staff.

refreshAllViews()

Master re-render trigger. Sequentially executes renderKPIs(), renderPOSProducts(), populateProductDropdown(), updateCheckoutSelectedProductUI(), renderInventoryTable(), renderHistoryTable(), and renderAnalyticsSummary().

renderKPIs()

Fetches latest metrics from getSalesSummary() and populates the 4 dashboard KPI cards. Dynamically applies red alert styling if low-stock count > 0.

renderPOSProducts()

Generates interactive visual product cards for the POS view with real-time stock badges ('In Stock', 'Low Stock', 'Out of Stock') based on search text and category filters.

populateProductDropdown()

Populates the `` dropdown in the checkout form with current catalog items, unit prices, and stock indicators.

selectProduct(productId)

Sets the globally active selectedProductId, highlights the selected card in the catalog grid, synchronizes the dropdown, and updates checkout form previews.

updateCheckoutSelectedProductUI()

Renders the selected item's name, category, SKU, live unit price box, live available stock count, and stock threshold badge inside the checkout panel.

updateLiveCalculation()

Computes and displays the live Total Amount before checkout (Price × Quantity). Displays an inline amber alert if the entered quantity exceeds currently available stock.

handleProcessCheckout()

Validates input and dispatches processCheckout() to the service layer. On success, displays a toast notification, triggers the receipt modal, and resets quantity to 1. On rejection, displays clear error feedback.

handleTestOversell()

Demonstration helper for college evaluators. Automatically populates the quantity field with (available stock + 5), allowing one-click testing of FR-10 oversell rejection.

showReceiptModal(transaction, remainingStock)

Populates and opens the Bootstrap Receipt Modal with transaction ID, cashier ID, item details, amount paid, and updated remaining stock.

renderInventoryTable()

Populates the Product Inventory Management table. Applies row-level warning backgrounds (row-low-stock) and badges. Includes Edit/Restock and Delete action buttons.

openAddProductModal()

Checks that the active role is Store Operations Staff (blocking cashiers). Resets form fields, auto-generates the next sequential SKU (e.g. P005), and shows the Add Product modal.

openEditProductModal(productId)

Checks Store Staff authorization. Loads existing product data into modal fields with read-only Product ID, allowing safe modification of price, stock, category, or thresholds.

handleProductFormSubmit(e)

Intercepts form submission, parses numeric values, and calls addProduct() or updateProduct() in the service layer. Handles error display or closes modal on success.

handleDeleteProduct(productId)

Prompts for confirmation and deletes a product from inventory (Store Staff only).

renderHistoryTable()

Renders the historical table of completed transactions with date/time, product SKU, cashier, quantity, unit price, total, and status badges.

renderAnalyticsSummary()

Computes and populates category-by-category inventory stock and revenue share progress bars in the Analytics tab.

showToast(title, message, type)

Creates and displays animated Bootstrap 5 floating toast notifications for success, warning, or error events.

escapeHtml(str)

Security utility to sanitize text strings before rendering into innerHTML, preventing Cross-Site Scripting (XSS).

4.3 Python Replay Script Functions (replay_checkout.py)

The Stretch Goal Python script demonstrates backend verification, atomic inventory reduction, and assertion checking.

load_data(filepath)

Reads sample-data.json from disk, parses products and transactions, and validates JSON file integrity.

run_replay(custom_transactions=None)

Main sequential transaction replay engine:
• Loads products into an in-memory dictionary keyed by productId.
• Replays checkout transactions one-by-one in sequential order.
• Verifies product existence and validates positive integer quantity.
• Compares requested quantity against available stock.
• If quantity > stock: Rejects transaction, logs reason, and leaves stock unchanged.
• If stock sufficient: Deducts stock, adds to total sales, and records successful transaction.
• **Assertion Check:** Executes assert product['stock'] >= 0 after every transaction.
• Prints a complete terminal audit report detailing successful transactions, rejected transactions, and final inventory.

5. Data Models & Entity Schemas

The application models two primary entities: **Product** and **Transaction**, formatted as standard JSON objects.

5.1 Product Entity Schema

Field	Type	Example	Description
productId	String	"P001"	Unique catalog SKU identifier.
name	String	"Notebook"	Display name of the product.
category	String	"Stationery"	Category (Stationery, Snacks, Accessories, etc.).
price	Number	60.00	Selling price per single unit in INR.
stock	Number	50	Current available inventory count.
lowStockThreshold	Number	10	Threshold at/below which low-stock highlight activates.
createdAt	String	"2026-10-30T09:00:00Z"	ISO timestamp of product catalog creation.
updatedAt	String	"2026-10-30T10:30:00Z"	ISO timestamp of last price or stock update.

5.2 Transaction Entity Schema

Field	Type	Example	Description
transactionId	String	"T001"	Unique sequential transaction identifier.
productId	String	"P001"	Reference SKU to purchased product.
productName	String	"Notebook"	Product name at purchase (preserves historical record).
quantity	Number	2	Number of units purchased in transaction.
unitPrice	Number	60.00	Unit price at time of purchase.
totalAmount	Number	120.00	Total amount paid (quantity × unitPrice).
cashierId	String	"C001"	Identifier of cashier who processed checkout.
createdAt	String	"2026-10-30T10:30:00Z"	ISO timestamp of checkout completion.
status	String	"SUCCESS"	Transaction status ('SUCCESS' or 'REJECTED').

6. Evaluator Step-by-Step Testing Guide

Follow these quick demonstration scenarios to test and grade every required capability:

Test 1: Normal Cashier Checkout (FR-04 to FR-09)

1. Ensure Role is set to **Cashier (C001)** in top navbar.
2. Select **Notebook (P001)**. Unit price ■60.00 and stock (50) display automatically.
3. Enter quantity **2**. Notice live total updates to ■**120.00**.
4. Click **'Complete Checkout'**.
5. Pop-up receipt appears showing order confirmation. Stock decrements to **48** immediately.

Test 2: Oversell Prevention Verification (FR-06 & FR-10)

1. Select **Water Bottle (P003)** (Stock: 8).
2. Click the **'Demo: Test Oversell Scenario'** button (sets quantity to 13).
3. Click **'Complete Checkout'**.
4. System rejects purchase with clear red alert: *'Insufficient stock available. (Available: 8, Requested: 13)'*.
5. Check inventory: stock remains safely untouched at **8 units**!

Test 3: Role Security Enforcement in JavaScript

1. While in **Cashier** mode, switch to the **Product Inventory** tab and click **'Add New Product'**.
2. The JavaScript logic blocks the modal and fires a red permission alert.
3. Switch role to **Store Operations Staff (S001)**.
4. Now click **'Add New Product'**: the modal opens cleanly and allows adding items.
5. Switch to **Checkout / POS** tab and click checkout: blocked with permission alert (Cashier only).

Test 4: Low-Stock Visual Highlighting (FR-11)

1. Notice **Water Bottle (P003)** has stock 8 and threshold 10.
2. It features an amber warning tag, animated pulsing indicator, and highlighted yellow row.
3. Low Stock Alert KPI card at top highlights in red.

Test 5: Run Python Sequential Replay Script (Stretch Goal)

Open terminal in the project directory and run:
python3 replay_checkout.py
The script replays transactions sequentially, verifies stock decrements, asserts assert product['stock'] >= 0, and outputs a formatted audit summary table.